

TASK 4: CUSTOMER ORDER HISTORY AND ORDER MANAGEMENT SYSTEM

Student Details:

- **Student Name:** Divya. V
- **Roll Number:** E25AI111
- **Department:** B.Sc Computer Science With AI
- **Project Title:** THE ARTISAN CRUMB: Bakery & Pastry Management System

1. Scenario Context & Operational Overview

The Order Management System serves as the primary transactional backbone for "THE ARTISAN CRUMB". It bridges customer profile data with specific product catalog purchases by separating transactions into normalized master order records (Orders) and individual line-item entries (Order_Details).

This architecture guarantees referential integrity, supports real-time status updates across the 6-stage order lifecycle, enables granular line-item tracking, and provides full financial auditability for revenue analysis.

2. Table Design & DDL Creation

Create Master Orders Table

```
SQL> CREATE TABLE Orders (  
2     Order_ID      VARCHAR2(10),  
3     Customer_ID   VARCHAR2(20),  
4     Order_Date    DATE,  
5     Total_Amount  NUMBER(10,2),  
6     Order_Status  VARCHAR2(20)  
7 );
```

Table created.

Create Order Details Transaction Table

```
SQL> CREATE TABLE Order_Details (  
2     Order_Detail_ID VARCHAR2(10),  
3     Order_ID         VARCHAR2(10),  
4     Product_ID       VARCHAR2(7),  
5     Quantity         NUMBER(5),  
6     Price            NUMBER(9,2)  
7 );
```

Table created.

3. Relationships & Constraints Definition

Primary Key Constraints

```
SQL> ALTER TABLE Orders
  2      ADD CONSTRAINT pk_orders PRIMARY KEY (Order_ID);

Table altered.

SQL>
SQL> ALTER TABLE Order_Details
  2      ADD CONSTRAINT pk_order_details PRIMARY KEY (Order_Detail_ID);

Table altered.
```

Foreign Key Constraints

```
SQL> ALTER TABLE Orders
  2      ADD CONSTRAINT fk_orders_customer FOREIGN KEY (Customer_ID)
  3      REFERENCES customer(customer_id);

Table altered.

SQL>
SQL> ALTER TABLE Order_Details
  2      ADD CONSTRAINT fk_od_orders FOREIGN KEY (Order_ID)
  3      REFERENCES Orders(Order_ID);

Table altered.

SQL>
SQL> ALTER TABLE Order_Details
  2      ADD CONSTRAINT fk_od_product FOREIGN KEY (Product_ID)
  3      REFERENCES Product(Product_ID);

Table altered.
```

Mandatory NOT NULL Constraints

```
SQL> ALTER TABLE Orders
  2      MODIFY Order_Date DATE NOT NULL;

Table altered.

SQL>
SQL> ALTER TABLE Order_Details
  2      MODIFY Quantity NUMBER(5) NOT NULL;

Table altered.
```

Business Logic CHECK Constraints

```
SQL> ALTER TABLE Orders
  2      ADD CONSTRAINT chk_order_status
  3      CHECK (Order_Status IN ('Pending', 'Baking', 'Ready for Pickup', 'Delivered', 'Cancelled'));

Table altered.

SQL>
SQL> ALTER TABLE Order_Details
  2      ADD CONSTRAINT chk_od_quantity CHECK (Quantity > 0);

Table altered.

SQL>
SQL> ALTER TABLE Order_Details
  2      ADD CONSTRAINT chk_od_price CHECK (Price > 0);

Table altered.
```

4. Data Population & Insertion

Interactive SQL Insertion Template for Orders Header

```
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Order_Status)
2 VALUES ('&ord_id', '&cust_id', TO_DATE('&ord_date', 'YYYY-MM-DD'), &total_amt, '&status');
Enter value for ord_id: ORD-501
Enter value for cust_id: CUST-101
Enter value for ord_date: 2026-03-18
Enter value for total_amt: 660.00
Enter value for status: Delivered
old 2: VALUES ('&ord_id', '&cust_id', TO_DATE('&ord_date', 'YYYY-MM-DD'), &total_amt, '&status')
new 2: VALUES ('ORD-501', 'CUST-101', TO_DATE('2026-03-18', 'YYYY-MM-DD'), 660.00, 'Delivered')

1 row created.

SQL> /
Enter value for ord_id: ORD-502
Enter value for cust_id: CUST-102
Enter value for ord_date: 2026-03-19
Enter value for total_amt: 850.00
Enter value for status: Baking
old 2: VALUES ('&ord_id', '&cust_id', TO_DATE('&ord_date', 'YYYY-MM-DD'), &total_amt, '&status')
new 2: VALUES ('ORD-502', 'CUST-102', TO_DATE('2026-03-19', 'YYYY-MM-DD'), 850.00, 'Baking')

1 row created.

SQL> /
Enter value for ord_id: ORD-503
Enter value for cust_id: CUST-103
Enter value for ord_date: 2026-03-20
Enter value for total_amt: 360.00
Enter value for status: Ready for Pickup
old 2: VALUES ('&ord_id', '&cust_id', TO_DATE('&ord_date', 'YYYY-MM-DD'), &total_amt, '&status')
new 2: VALUES ('ORD-503', 'CUST-103', TO_DATE('2026-03-20', 'YYYY-MM-DD'), 360.00, 'Ready for Pickup')

1 row created.

SQL> /
Enter value for ord_id: ORD-504
Enter value for cust_id: CUST-101
Enter value for ord_date: 2026-03-22
Enter value for total_amt: 510.00
Enter value for status: Pending
old 2: VALUES ('&ord_id', '&cust_id', TO_DATE('&ord_date', 'YYYY-MM-DD'), &total_amt, '&status')
new 2: VALUES ('ORD-504', 'CUST-101', TO_DATE('2026-03-22', 'YYYY-MM-DD'), 510.00, 'Pending')

1 row created.
```

Interactive SQL Insertion Template for Order Line Items

```
SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES ('OD-01', 'ORD-501', 'BAK-101', 2, 270.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES ('OD-02', 'ORD-502', 'BAK-102', 1, 850.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES ('OD-03', 'ORD-503', 'BAK-103', 2, 180.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES ('OD-04', 'ORD-504', 'BAK-101', 1, 270.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES ('OD-05', 'ORD-501', 'BAK-104', 1, 120.00);

1 row created.

SQL>
SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
  2 VALUES ('OD-06', 'ORD-504', 'BAK-104', 2, 120.00);

1 row created.
```

5. Table Display Queries

Select all records from Orders table

```
SQL> SELECT Order_ID, Customer_ID, TO_CHAR(Order_Date, 'YYYY-MM-DD') AS Order_Date, Total_Amount, Order_Status
  2 FROM Orders;
```

ORDER_ID	CUSTOMER_ID	ORDER_DATE	TOTAL_AMOUNT	ORDER_STATUS
ORD-501	CUST-101	2026-03-18	660	Delivered
ORD-502	CUST-102	2026-03-19	850	Baking
ORD-503	CUST-103	2026-03-20	360	Ready for Pickup
ORD-504	CUST-101	2026-03-22	510	Pending

Select all records from Order_Details table

```
SQL> SELECT Order_Detail_ID, Order_ID, Product_ID, Quantity, Price
2 FROM Order_Details;
```

ORDER_DETA	ORDER_ID	PRODUCT	QUANTITY	PRICE
OD-01	ORD-501	BAK-101	2	270
OD-02	ORD-502	BAK-102	1	850
OD-03	ORD-503	BAK-103	2	180
OD-04	ORD-504	BAK-101	1	270
OD-05	ORD-501	BAK-104	1	120
OD-06	ORD-504	BAK-104	2	120

6 rows selected.

6. Advanced Analytical Reports & DQL Queries

Report 1: Complete Customer Order History

```
SQL> SELECT c.customer_id,
2         c.first_name || ' ' || c.last_name AS customer_fullname,
3         c.city,
4         o.Order_ID,
5         TO_CHAR(o.Order_Date, 'YYYY-MM-DD') AS order_date,
6         o.Total_Amount,
7         o.Order_Status
8 FROM customer c
9 JOIN Orders o ON c.customer_id = o.Customer_ID
10 ORDER BY o.Order_Date DESC;
```

CUSTOMER_ID	CUSTOMER_FULLNAME
CITY	ORDER_ID ORDER_DATE TOTAL_AMOUNT ORDER_STATUS
CUST-101 Chennai	Siddharth Sharma ORD-504 2026-03-22 510 Pending
CUST-103 Coimbatore	Arjun Das ORD-503 2026-03-20 360 Ready for Pickup
CUST-102 Bengaluru	Kavya Patel ORD-502 2026-03-19 850 Baking

CUSTOMER_ID	CUSTOMER_FULLNAME
CITY	ORDER_ID ORDER_DATE TOTAL_AMOUNT ORDER_STATUS
CUST-101 Chennai	Siddharth Sharma ORD-501 2026-03-18 660 Delivered

Report 2: Product-Wise Sales Volume Report

```
SQL> SELECT p.Product_ID,  
2      p.Title AS Product_Name,  
3      COUNT(od.Order_Detail_ID) AS Times_Ordered,  
4      SUM(od.Quantity) AS Units_Sold,  
5      SUM(od.Quantity * od.Price) AS Total_Sales_Revenue  
6 FROM Product p  
7 JOIN Order_Details od ON p.Product_ID = od.Product_ID  
8 GROUP BY p.Product_ID, p.Title  
9 ORDER BY Total_Sales_Revenue DESC;
```

PRODUCT

PRODUCT_NAME

TIMES_ORDERED UNITS_SOLD TOTAL_SALES_REVENUE

BAK-102

Chocolate Fudge Cake

1 1 850

BAK-101

Sourdough Bread

2 3 810

PRODUCT

PRODUCT_NAME

TIMES_ORDERED UNITS_SOLD TOTAL_SALES_REVENUE

BAK-103

Almond Croissant

1 2 360

BAK-104

Red Velvet Cupcake

PRODUCT

PRODUCT_NAME

TIMES_ORDERED UNITS_SOLD TOTAL_SALES_REVENUE

2 3 360

Report 3: Customer Purchase & Behavioral Analysis

3a. Top spending customer using nested inline subquery

```
SQL> SELECT customer_id, customer_name, Total_Spent
2 FROM (
3     SELECT c.customer_id,
4           c.first_name || ' ' || c.last_name AS customer_name,
5           SUM(o.Total_Amount) AS Total_Spent
6     FROM customer c
7    JOIN Orders o ON c.customer_id = o.Customer_ID
8   GROUP BY c.customer_id, c.first_name, c.last_name
9   ORDER BY Total_Spent DESC
10 )
11 WHERE ROWNUM = 1;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_SPENT
CUST-101	Siddharth Sharma	1170

3b. Total cumulative spending per registered customer

```
SQL> SELECT c.customer_id,
2       c.first_name || ' ' || c.last_name AS customer_fullname,
3       NVL(SUM(o.Total_Amount), 0) AS Cumulative_Expenditure
4 FROM customer c
5 LEFT JOIN Orders o ON c.customer_id = o.Customer_ID
6 GROUP BY c.customer_id, c.first_name, c.last_name;
```

CUSTOMER_ID	CUSTOMER_FULLNAME	CUMULATIVE_EXPENDITURE
CUST-102	Kavya Patel	850
CUST-103	Arjun Das	360
CUST-101	Siddharth Sharma	1170

CUSTOMER_ID	CUSTOMER_FULLNAME	CUMULATIVE_EXPENDITURE
CUST-104	Meera Iyer	0

3c. Overall Average Order Value

```
SQL> SELECT ROUND(AVG(Total_Amount), 2) AS Average_Order_Value  
2 FROM Orders;
```

AVERAGE_ORDER_VALUE
595